José anibal Cayetano meza

DOCUMENTO EXPLICATIVO

SECCIÓN 1 – PARTE A

1. Cuando conviene mover logica de PHP a Stored Procedures y cuando NO?
   - Si. conviene: Cuando las reglas del negocio son criticas y son compartidas por varias aplicaciones, para reducir el pull e conexiones a la base datos.
   - No conviene: cuanto el testeo es dificil, cuando el equipo no tiene experiencia en base datos.

2. Diferencia entre vista normal y vista materializada. Como simular una materializada en MySQL?
   - **Normal**: solo guarda la query, se ejecuta cada vez → resultados siempre frescos pero costosos.
   - **Materializada**: guarda datos persisentes y se refresca bajo demanda → más rápido en lecturas pesadas.
   - **MySQL/MariaDB (simulación)**: tabla cache (mv_*) + job/event scheduler o proceso externo que refresque; triggers para mantener incrementalmente.

3. Explica como evitar condiciones de carrera (race conditions) dentro de un SP.
   - Usar **transacciones** (START TRANSACTION) con aislamiento adecuado.
   - Evitar el patrón inseguro "check-then-insert" sin locks.

4. Como implementar idempotencia en procedimientos que generan documentos fiscales?
   - Constraint UNIQUE para evitar duplicados.

5. Como optimizar un RANK() o ranking en MySQL sin afectar rendimiento masivo?
   - No hacer ranking sobre datasets completos sin filtros.
   - Limitar ranking por sección/periodo.
   - Índices en (section_id, promedio DESC).

6. Estrategia para evitar full table scans en reportes academicos de 10k estudiantes.
   - Evitar funciones sobre columnas indexadas en WHERE
   - Paginación eficiente (keyset pagination).

7. Manejo correcto de NULL vs '' en columnas INT y DECIMAL.
   - **NULL** = ausencia de valor
   - '' en numéricos fuerza cast y rompe índices.
   - Usar NULL para "no aplica".

8. Como estructurar auditoria SQL robusta para facturacion.

- Guardar cambios en JSON.

9. Como manejar collation conflicts en un sistema multi base de datos.
   - **Estandarizar en utf8mb4 y collation única.**
   - **Evitar mezclar latin1 y utf8mb4.**

10. Estrategia para versionar procedimientos en produccion sin romper el sistema.
    - **Registrar versión en tabla schema_version.**
    - **Estrategia** blue/green**: sp_x_v2 paralelo a sp_x_v1.**

SECCIÓN 2 – Diseño Arquitectónico General

Objetivo

- Unificar lógica duplicada de cálculos (promedios).
- Garantizar regla fiscal de 7 días (bajas vs notas de crédito).
- Optimizar dataset de libretas.

- Eliminar full scans en reportes académicos.

Principios

- **DRY en SQL**: una sola fuente de verdad.
- **Transacciones ACID**: consistencia garantizada.
- **Idempotencia**: control por request_id.
- **Auditoría completa**: trazabilidad de cambios.
- **Performance**: índices compuestos + cache por clave.

Estrategia

- Vistas**: consolidación de lectura.**
- Stored Procedures (SP)**: reglas críticas y operaciones de escritura.**
- Funciones**: cálculos reutilizables.**
- Tablas auxiliares**:**
- **fe_audit_log para trazabilidad.**
- **pdf_cache para cache de documentos.**

SECCIÓN 3 – Problema 1 (Promedios)

Entregables

- **Vista** view_unidad_promedio: consolida notas por unidad y peso, evitando duplicidad de joins en reportes.
- **Función** fn_promedio_ponderado(area_id, student_id): centraliza el cálculo ponderado, eliminando lógica duplicada en PHP/SQL.
- **SP** sp_get_area_unit_and_annual_grades(student_id): obtiene calificaciones por área, unidad y anual.
- **Ranking optimizado**: sobre dataset pre-agregado (sección + examen + área) para reducir costo.

Optimización

- Índices compuestos **orientados a filtros: (section_id, exam_id, area_id, unit_id, student_id)**
- **Índices de apoyo: (student_id, exam_id).**

SECCIÓN 4 – Problema 2 (Facturación 7 días)

Entregables

- **Vista** view_document_status: estado consolidado de documentos.
- **SP** sp_process_void_request(document_id): procesa solicitudes de baja.
- **SP** sp_generate_credit_note(document_id): genera nota de crédito automática.
- **SP** sp_normalize_invoice_data(): normaliza y estandariza datos de facturas.
- **Tabla** fe_audit_log: registra auditoría completa.

Regla fiscal

- Si DATEDIFF(CURDATE(), issue_date) > 7 y se solicita baja → se genera nota de crédito **automática.**

Consistencia

- Uso de transacción con SELECT … FOR UPDATE para asegurar integridad del documento.

Auditoría

- Cada cambio inserta registro en fe_audit_log con request_id → trazabilidad e idempotencia.

Normalización

- Limpieza y estandarización de campos.
- Marcado de registros para evitar reprocesos.

José anibal Cayetano meza

SECCIÓN 5 – Problema 3 (Libretas + Cache)

Entregables

- Vista `view_libreta_dataset`: dataset agregado por alumno/área, simplifica generación de PDFs.
- SP `sp_generate_libreta_dataset(section_id, exam_id)`: genera dataset consolidado por sección y examen.
- Tabla `pdf_cache`: guarda payload por clave (`LIBRETA:section:exam`) con TTL.
- Invalidación: al cambiar notas de una sección/examen se invalida el `cache_key` (por trigger o aplicación).

Optimización

Índices:

- PK en `pdf_cache(cache_key)`.
- Índice compuesto `(section_id, exam_id, valid_until)` para consultas por lote.

SECCIÓN 6 – Problema 4 (Optimización avanzada)

```
EXPLAIN
SELECT ug.student_id,
SUM(COALESCE(ug.grade,0) * u.weight) / SUM(u.weight) AS avg_grade
FROM unit_grade ug
JOIN unit u ON u.id = ug.unit_id
WHERE ug.section_id = 1 AND ug.exam_id = 1 AND ug.area_id = 1
GROUP BY ug.student_id
ORDER BY avg_grade DESC;
```

id	select_type	table	type	possible_keys	key	key_len	ref	rows	Extra
1	SIMPLE	NULL	NULL	NULL	NULL	NULL	NULL	NULL	Impossible WHERE noticed after reading const tables

```
EXPLAIN
SELECT * FROM fe_document
WHERE issue_date BETWEEN '2026-01-01' AND '2026-12-31'
  AND status IN ('ACTIVE','VOIDED');
```

id	select_type	table	type	possible_keys	key	key_len	ref	rows	Extra
1	SIMPLE	fe_document	range	ix_doc_issue_status	ix_doc_issue_status	4	NULL	1	Using index condition

```
EXPLAIN
SELECT payload FROM pdf_cache
WHERE cache_key = 'LIBRETA:1:1'
  AND valid_until > NOW();
```

id	select_type	table	type	possible_keys	key	key_len	ref	rows	Extra
1	SIMPLE	NULL	NULL	NULL	NULL	NULL	NULL	NULL	Impossible WHERE noticed after reading const tables